Problem 1: Recursive Method to calculate the sum from 1 to n

- Step 1: Should return 0 when n is less than or equal to 0
- Step 2: should return 1 when n equals 1
- Step 3: recursive case should return n + sum(n - 1)

Problem 2: Write a recursive method to calculate the sum from a to b

- Step 1: base case: return b if a >= b
- Step 2: recursive case should return a + sum (a + 1, b)

Problem 3: Write a recursive method to find the minimum value in the array

- Step 1: if n is 1, should return the only value in the array
- Step 2: recursive case needs to return the min value in the last value of the array, compared to the rest of the array

Problem 4: Write a recursive method to calculate the sum of all elements in the array

- Step 1: if n is 0 or a negative number, should return 0 as sum
- Step 2: recursive case needs to loop in a way that shrinks the array by 1 while adding the value to n

Problem 5: Helper method for binary search

- Step 1: if left is greater than right, return -1, as the array is sorted
- Step 2: create the mid using the difference between left and right
- Step 3: Best case: if arr[mid] == key, return mid
- Step 4: recursive case: search the half where the key could be
- Step 5: branch whether right or left side of array. Repeat to shrink the array size till key is found

<u>Problem Breakdown</u>

1. The base case stops the method, and the recursive case keeps reducing n until it reaches that point.
2. The base case stops when the range is empty, and the recursive case keeps moving through the range.
3. The base case happens when only one item is left, and the recursive case keeps shrinking the array.
4. The base case stops when no elements are left, and the recursive case keeps adding the rest.
5. The base case stops when the key is not found, and the recursive case keeps checking the correct half.